In [1]:
```
import sys, os
import numpy as np
from typing import List, Optional

assignment_root = os.path.abspath(os.getcwd())
if assignment_root not in sys.path:
    sys.path.insert(0, assignment_root)
print("Added to sys.path:", assignment_root)

from fixedincomelib import *
print("Fixed Income Library is loaded.")
```

```
Added to sys.path: /Users/syonamehra/Desktop/FRE-GY-9743-Assignment-1
Fixed Income Library is loaded.
```

Homework 1 --- 1-D Interpolation

Implement the four methods marked `## TODO` inside `Interpolator1DPCP`, in `fixedincomelib/utilities/numerics.py`:

- `interpolate`
- `integrate`
- `gradient_wrt_ordinate`
- `gradient_of_integrated_value_wrt_ordinate`

Then fill in `bump_reval_interpolator_integrand` further down in this notebook.

The interpolation convention is spelled out in the `Interpolator1DPCP` docstring. Read it before writing.

To check yourself, run every cell in this notebook top to bottom. Each check prints your value next to the expected one --- every `diff` should be around `0.0`.

Test interpolation

In [2]:
```
axis1 = [1, 3, 5, 7]
values = [3, 4, 5, 6]
interp_method = 'PIECEWISE_CONSTANT_LEFT_CONTINUOUS'
extrap_method = 'FLAT'
interp_1d = qfCreate1DInterpolator(axis1, values, interp_method, extrap_meth

test_points = [
    (0.5, 3.0),   # left wing, flat extrapolation
    (1.0, 3.0),   # exactly on the first node
    (1.5, 4.0),   # inside (1, 3]
    (3.0, 4.0),   # exactly on an interior node
    (5.5, 6.0),   # inside (5, 7]
    (6.5, 6.0),   # inside (5, 7]
    (8.0, 6.0),   # right wing, flat extrapolation
]
```

```
for x, expected in test_points:
    v = qfInterpolate1D(x, interp_1d)
    print(f'f({x}) = {v}, expected {expected}, diff = {v - expected}')
```

```
f(0.5) = 3, expected 3.0, diff = 0.0
f(1.0) = 3, expected 3.0, diff = 0.0
f(1.5) = 4, expected 4.0, diff = 0.0
f(3.0) = 4, expected 4.0, diff = 0.0
f(5.5) = 6, expected 6.0, diff = 0.0
f(6.5) = 6, expected 6.0, diff = 0.0
f(8.0) = 6, expected 6.0, diff = 0.0
```

Test integration of the interpolation

Both endpoints may land anywhere: inside a bucket, on a node, or out in either flat wing.

In [3]:
```
integration_cases = [
    ((0.5, 0.9),   1.2),   # both inside the left wing
    ((0.5, 1.2),   2.3),   # left wing into the first bucket
    ((0.5, 3.2),  10.5),   # left wing across into the middle
    ((1.5, 5.2),  17.2),   # entirely inside the node range
    ((3.5, 7.2),  20.7),   # middle bucket out into the right wing
    ((6.0, 7.2),   7.2),   # last bucket into the right wing
    ((8.0, 10.0), 12.0),   # both inside the right wing
    ((0.1, 10.0), 50.7),   # spanning everything
]

for (x_s, x_e), expected in integration_cases:
    v = qfInterpolate1DIntegral(x_s, x_e, interp_1d)
    print(f'integral over [{x_s}, {x_e}] = {v}, expected {expected}, diff =
```

```
integral over [0.5, 0.9] = 1.2000000000000002, expected 1.2, diff = 2.220446
049250313e-16
integral over [0.5, 1.2] = 2.3, expected 2.3, diff = 0.0
integral over [0.5, 3.2] = 10.5, expected 10.5, diff = 0.0
integral over [1.5, 5.2] = 17.200000000000003, expected 17.2, diff = 3.55271
3678800501e-15
integral over [3.5, 7.2] = 20.700000000000003, expected 20.7, diff = 3.55271
3678800501e-15
integral over [6.0, 7.2] = 7.200000000000001, expected 7.2, diff = 8.8817841
97001252e-16
integral over [8.0, 10.0] = 12.0, expected 12.0, diff = 0.0
integral over [0.1, 10.0] = 50.7, expected 50.7, diff = 0.0
```

Sensitivities

Implement the two analytic sensitivity methods so that they agree with a bump-and-reval reference:

- `gradient_wrt_ordinate`
- `gradient_of_integrated_value_wrt_ordinate`

`bump_reval_interpolator` below is a worked bump-and-reval for the interpolated value. Mirror its structure to fill in `bump_reval_interpolator_integrand` for the integral, then contrast both against your analytic results.

In [4]:
```python
def bump_reval_interpolator(
    x : float,
    axis1 : List,
    values : List,
    interp_method : str,
    extrap_method : str,
    bump_size : Optional[float] = 1e-4):

    base_interpolator = qfCreate1DInterpolator(axis1, values, interp_method,
    b_value = qfInterpolate1D(x, base_interpolator)

    grad = []
    for i in range(len(values)):
        values[i] += bump_size
        this_interp = qfCreate1DInterpolator(axis1, values, interp_method, e
        bumped_value = qfInterpolate1D(x, this_interp)
        grad.append((bumped_value - b_value) / bump_size)
        values[i] -= bump_size

    return np.array(grad)


def bump_reval_interpolator_integrand(
    x_s : float,
    x_e : float,
    axis1 : List,
    values : List,
    interp_method : str,
    extrap_method : str,
    bump_size : Optional[float] = 1e-4):

    base_interpolator = qfCreate1DInterpolator(axis1, values, interp_method,
    b_value = qfInterpolate1DIntegral(x_s, x_e, base_interpolator)

    grad = []
    for i in range(len(values)):
        values[i] += bump_size
        this_interp = qfCreate1DInterpolator(axis1, values, interp_method, e
        bumped_value = qfInterpolate1DIntegral(x_s, x_e, this_interp)
        grad.append((bumped_value - b_value) / bump_size)
        values[i] -= bump_size

    return np.array(grad)
```

Interpolation sensitivity

In [5]:
```python
for x, _ in test_points:
    grad_analytic = qfInterpolate1DGrad(x, interp_1d)
```

```
    grad_br = bump_reval_interpolator(x, axis1, values, interp_method, extra
    print(f'x = {x}: max abs diff = {np.max(np.abs(grad_analytic - grad_br))
```

```
x = 0.5: max abs diff = 2.1103119252074976e-12
x = 1.0: max abs diff = 2.1103119252074976e-12
x = 1.5: max abs diff = 2.1103119252074976e-12
x = 3.0: max abs diff = 2.1103119252074976e-12
x = 5.5: max abs diff = 2.3305801732931286e-12
x = 6.5: max abs diff = 2.3305801732931286e-12
x = 8.0: max abs diff = 2.3305801732931286e-12
```

Integrated interpolation sensitivity

In [6]:
```
for (x_s, x_e), _ in integration_cases:
    grad_analytic = qfInterpolate1DIntegralGrad(x_s, x_e, interp_1d)
    grad_br = bump_reval_interpolator_integrand(
        x_s, x_e, axis1, values, interp_method, extrap_method)
    print(f'[{x_s}, {x_e}]: max abs diff = {np.max(np.abs(grad_analytic - gr
```

```
[0.5, 0.9]: max abs diff = 4.000133557724439e-13
[0.5, 1.2]: max abs diff = 1.3102852136626097e-12
[0.5, 3.2]: max abs diff = 7.571721027943568e-12
[1.5, 5.2]: max abs diff = 2.7955415760061442e-11
[3.5, 7.2]: max abs diff = 2.1259438653942198e-11
[6.0, 7.2]: max abs diff = 1.0205170042354439e-12
[8.0, 10.0]: max abs diff = 4.661160346586257e-12
[0.1, 10.0]: max abs diff = 1.1823431123048067e-10
```

Bond Forward Business: Written Answers

Notation from the assignment sheet: trade date `0`, settlement date `Ts` (1 year out), bond maturity `Tm`, forward strike `K`, spot bond price `B(0;0,Tm)`, forward bond price `B(0;Ts,Tm)`, repo rate `R` for the period `[0,Ts]`.

Q1: Why would the client want this, what's on the contract, what happens at settlement

The main reason a client does this trade is to lock in a price now for something they need later. If a client knows they'll have cash in a year and plan to buy this bond with it, they're worried rates (and so bond prices) will move against them before then. A forward fixes the price today, and nobody pays anything upfront.

Other reasons a client might do this: they already hold a bond portfolio and want to change how sensitive it is to rates without actually trading the cash bond (which moves the market and has tax/accounting consequences), or they just have a view that rates will fall and want to bet on it without paying the full price of the bond today, since a forward only needs margin.

The contract needs to spell out: the notional ($1,000,000), exactly which bond (CUSIP, coupon, maturity `Tm`), the settlement date `Ts`, the agreed price `K`, whether it settles physically or in cash, and the usual legal stuff (governing ISDA, the CSA/collateral terms, who the two sides are).

At settlement, Bank A hands the client the bond, and the client pays `K`. That's really all equation (1) is saying: the client ends up owning a bond worth whatever it's worth that day, `B(Ts;Ts,Tm)`, but only paid the price agreed a year earlier.

Q2: Why buy the bond right away, and how the desk pays for it

The second the desk agrees to sell the client a bond at `K` a year from now, it has taken on risk. If bond prices go up before then, the desk would need to buy an expensive bond later just to hand it over for the old, cheaper price. To get rid of that risk, the desk simply buys the actual bond today and holds it for the year. Now it doesn't matter which way prices move, because the desk already owns exactly what it has to deliver.

Buying the bond costs money, though, and the desk doesn't want to tie up a million dollars of the bank's own cash for a year. So internally: the treasury desk lends the trading desk the cash to buy the bond, and the trading desk then uses that same bond as collateral to borrow cash back from the repo desk. This is basically a pawn shop transaction: you hand something over, get cash for it, pay some interest, and get it back later. That repo cash pays off the treasury loan. Any coupons the bond pays during the year still go to the desk, since it's still the real owner of the bond.

Q3: Getting the forward price from the spot price, coupons, and the repo rate

If you borrow $1 in repo at rate `R` for a year, you owe `(1+R)` back at the end. So if the desk borrowed `B(0;0,Tm)` (today's bond price) to fund the purchase, it owes `B(0;0,Tm)·(1+R)` a year later.

But the bond also pays out coupons during the year, and those get passed to the desk. That coupon money reduces how much the desk actually has to come up with at the end, so it gets subtracted (grown a bit too, since the desk could have reinvested it at the repo rate):

```
B(0;Ts,Tm) = B(0;0,Tm)·(1+R) − sum of [coupon_i · (1 +
R·(time left after coupon_i))]
```

In plain words: forward price = spot price grown at the repo rate, minus the coupons collected along the way (also grown at the repo rate).

Q4: What is the trade worth partway through the year

At any point `t` before `Ts`, you can ask what the trade is worth right now. You just redo the Q3 calculation with today's numbers instead of the original ones: today's bond price, today's repo rate for however much time is left, and whichever coupons are still left to be paid. That gives an updated forward price `B(t;Ts,Tm)`. Compare that to the locked-in `K`, and discount the difference back to today using whatever discount rate matches the collateral agreement (usually SOFR/OIS):

```
V(t) = discount_factor(t,Ts) · ( B(t;Ts,Tm) − K )
```

Q5: Does the desk actually make money here

If the desk hedges itself perfectly by buying the bond and financing it, it doesn't have any real price risk left. But that raises the obvious question: if there's no risk, how does the desk make money, since profit usually comes from taking on risk?

The answer is the desk doesn't need price risk to make money on this trade. It makes money two other ways:

- It quotes the client a slightly worse price than the true fair value, same as a store marking something up.
- Treasury bonds, especially the current 10-year, are often in high demand in the repo market, so the desk can sometimes borrow against the bond more cheaply than the "normal" rate it assumed when quoting the client (this is called the bond going "special" in repo). That gap is pure profit for the desk.

So the honest pitch to the client is that they're not paying the desk to bet on rates, they're paying the desk to take the financing and logistics hassle off their hands.

*Q5: How this connects to risk-neutral pricing

The Q3 formula came from just thinking through how you'd actually replicate the trade with a real bond and a real loan, no advanced math needed. It turns out this gives the same answer as the formal risk-neutral pricing approach, just reached a more hands-on way. In risk-neutral pricing terms, the forward price equals the spot price minus the value of the coupons, divided by a discount factor, and that discount factor works out to be `1/(1+R)`, the same `R` as the repo rate.

These two approaches only match exactly if the repo rate is a plain, risk-free borrowing rate: no specialness, known ahead of time, and no extra credit risk added on top. The repo specialness from Q5 is exactly the thing that breaks this clean match, and it's also exactly where the desk's profit was coming from.